

안녕하세요.
프론트엔드 개발자 유상록입니다.

전화번호

010-8269-8150

이메일

dbtkdfhr2000@naver.com

GitHub

<https://github.com/dbtkdfhr>



About Me

협업을 통해 배우는 것을 좋아합니다.

배울 점은 누구에게나 존재합니다. 협업을 진행하며 새로운 지식을 습득하고 내 것으로 만들며 성장하고자 노력하고 있습니다.

안주하지 않습니다.

현재의 코드나 방식에 만족하지 않고, 항상 더 효율적인 구조와 성능 개선을 고민합니다.

새로운 지식을 배우는데 망설이지 않습니다

새로운 기술 트렌드에 호기심이 많으며, 배운 것을 주도적으로 프로젝트에 적용하고 팀과 공유합니다.

Education

국민대학교 소프트웨어학과

2019.03 ~ 2025.02 졸업

Project

Schemafy 프론트엔드 개발 | 2025.08 -

기술 스택: React, TailwindCSS, MobX, PlayWright

팀 규모: 4명 (프론트 2, 백엔드 2)

프로젝트 개요

학기 중 FlowERD라고 하는 복잡한 데이터베이스 설계를 효율적으로 관리하기 위한 ERD 모델링 도구를 제작하였지만 많이 부족한 부분을 느껴, 졸업 후 Schemafy라는 이름을 다시 붙여 부족한 부분들을 보완하고자 다시 처음 설계부터 진행하기 시작하였습니다.

대용량 데이터 전송 효율성과 데이터 정합성 문제를 해결하기 위해 아키텍처를 전면 재설계하였으며, 안정적인 서비스 운영을 위한 테스트 자동화를 도입했습니다.

모노레포 기반 아키텍처 설계

FlowERD에서는 ERD 상태를 스냅샷 단위로 저장하는 구조를 사용하면서, 다이어그램이 복잡해질수록 통신 비용이 과도하게 증가하는 문제가 발생했습니다.

또한 프론트엔드와 백엔드가 동일한 ERD 도메인 로직을 각자 구현하면서 데이터 변조 가능성과 로직 불일치 위험이 존재한다고 판단했습니다.

이를 해결하기 위해 모노레포 구조를 도입하고, 프론트엔드·백엔드와 별도로 Validation 패키지를 분리하여 구성했습니다.

해당 패키지에는 ERD 구조 정의, 관계 규칙, 스키마 제약 등 도메인 핵심 로직을 단일한 기준으로 정리하였습니다.

프론트엔드와 백엔드는 동일한 Validation 패키지를 참조하여 동일한 로직을 동시에 검증하도록 구성하였고, Validation을 통과한 최소 단위의 데이터만을 전송하는 방식으로 통신 비용을 크게 절감할 수 있었습니다.

이를 통해 데이터 정합성과 성능을 동시에 확보한 구조를 설계할 수 있었습니다.

Playwright 기반 E2E 테스트 도입

ERD 도메인 로직을 Validation 패키지로 분리하면서 단위 테스트의 중요성을 체감하게 되었고, 이에 따라 프론트엔드 전체 흐름을 검증할 수 있는 E2E 테스트 도입의 필요성을 느끼게 되었습니다. Schemafy는 MVP 단계에서 기능 변경이 매우 잦았고, 그에 따라 수동 QA에 많은 시간이 소요되는 문제가 있었습니다.

이를 해결하기 위해 Playwright를 도입하여 주요 사용자 시나리오를 E2E 테스트로 자동화하였습니다.

ERD 생성, 테이블 수정, 관계 연결 등 핵심 플로우를 테스트로 고정함으로써 기능 변경 시 발생하던 회귀 버그를 빠르게 탐지할 수 있었고, QA 및 배포에 소요되는 시간을 크게 단축할 수 있었습니다.

API 추가, 수정이 진행되면서 적용하던 중에 네트워크 사용량에 이상을 감지했습니다.

확인해 보니 같은 API를 여러 번 요청하는 부분들이 많이 발견되었습니다.

SharedWorker 기반 실시간 협업 네트워크 최적화

Schemafy는 실시간 협업 도구 특성상 사용자가 동일한 프로젝트를 여러 탭에 띄워놓고 작업하는 경우가 빈번했습니다. 이 과정에서 각 탭마다 별도의 웹소켓 연결이 생성되어 불필요한 네트워크 리소스가 낭비되고 서버 부하가 가중되는 문제를 발견했습니다. 이를 해결하기 위해 SharedWorker를 도입하여 브라우저 탭 간에 단일 웹소켓 인스턴스를 공유하는 구조로 아키텍처를 고도화했습니다.

- **Dual Worker 전략 적용:** SharedWorker를 지원하지 않는 환경을 고려하여, 런타임 환경에 따라 DedicatedWorker로 자동 폴백(Fallback) 처리되도록 구현하여 서비스의 범용성을 확보했습니다.
- **연결 생명주기 관리:** SharedWorker 특성상 개별 탭의 종료 이벤트를 즉각 감지하기 어려운 한계를 극복하기 위해, Heartbeat 메커니즘을 자체 구현했습니다. 이를 통해 비활성 포트를 주기적으로 정리하고 메모리 누수를 원천 차단했습니다.
- **성과:** 다중 탭 환경에서 중복 연결을 1개로 통합함으로써 클라이언트의 데이터 사용량을 획기적으로 절감하고, 서버의 동시 접속 부하를 효율적으로 분산시켰습니다.

ERD 협업 에디터 Undo/Redo 설계 (실시간 다중 사용자 환경)

실시간 협업 환경에서 Undo/Redo를 안정적으로 동작시키기 위한 구조를 설계했습니다.

처음에는 프론트엔드에서 일반적인 Command 패턴으로 Undo/Redo를 관리하는 방식을 검토했지만, 이 방식은 협업 환경에 적합하지 않았습니다.

사용자 A가 되돌린 변경이 사용자 B의 화면에는 반영되지 않았고, 같은 사용자가 여러 탭에서 작업하는 경우에도 각 탭의 상태가 서로 동기화되지 않는 문제가 있었습니다.

이 문제를 해결하기 위해 클라이언트가 상태를 직접 되돌리는 방식이 아니라, 서버를 공유 상태의 단일 기준으로 두는 구조로 전환했습니다.

클라이언트는 최종 상태를 직접 수정하지 않고 “어떤 변경을 하려는지”를 operation으로 서버에 전달하도록 했습니다.

Undo/Redo 역시 이전 문서 상태로 롤백하는 방식이 아니라, 기존 변경의 반대 동작을 새로운 operation으로 발행하는 방식으로 모델링했습니다.

동시 편집 상황에서 충돌을 감지하기 위해 schema 단위 revision을 도입했습니다.

모든 변경 요청에는 클라이언트 작업 ID, 기존 revision, 협업 세션 ID를 함께 전달하도록 했고, 서버는 단일 트랜잭션 안에서 revision 검증, operation 적용, inverse operation 계산, 로그 저장, revision 증가를 순서대로 처리하도록 설계했습니다.

협업 세션 ID를 통해 같은 사용자의 다른 탭에서 발생한 변경도 구분할 수 있게 했고, 탭 단위 Undo/Redo 스택을 유지했습니다.

변경이 확정되면 서버는 WebSocket으로 전체 구독자에게 변경 사실을 브로드캐스트하고, 클라이언트는 영향을 받은 테이블만 다시 조회하도록 구성했습니다.

그 결과 사용자별 Undo/Redo 흐름을 유지하면서도 탭 간, 사용자 간 데이터 정합성을 보장할 수 있었습니다.

Project

Schemafy – BFF 서버 설계 및 구현 | 2025.08 -

기술 스택: NestJS, TypeScript, Axios, WebSocket, HMAC-SHA256

팀 프로젝트 (ERD 실시간 협업 툴)

프로젝트 개요

프론트엔드를 담당하던 중, 기존 구조의 보안 취약점을 발견하고 BFF 서버의 필요성을 팀에 제안하여 직접 설계 및 구현을 맡았습니다.

BFF 도입 배경 – 요청 출처를 신뢰할 수 없는 문제

기존 구조는 프론트엔드 클라이언트가 Spring Boot 백엔드에 직접 요청을 보내는 방식이었습니다. 백엔드에서 입력값 유효성 검사는 수행하지만, 유효성 조건을 만족하는 요청이라면 실제 클라이언트 애플리케이션에서 온 것인지, 악의적인 사용자가 직접 조작하여 보낸 것인지 구별할 방법이 없었습니다. 이를 해결하려면 백엔드가 검증된 출처의 요청만 신뢰하도록 만들어야 했고, HMAC 서명이 적합한 방법이었습니다.

그러나 HMAC 시크릿을 브라우저 클라이언트에 두면 번들 코드에서 탈취될 수 있어 서버 측에서만 보관해야 했습니다.

이 이유로 BFF를 중간 계층으로 두고, BFF만이 시크릿을 보유하여 백엔드로 나가는 모든 요청에 서명하는 구조를 설계했습니다.

HMAC 요청 서명 구현

BFF가 백엔드로 요청을 보낼 때, HTTP메서드 + 경로 + 타임스탬프 + Nonce +바디해시를 정규 문자열로 조합한 뒤 공유 시크릿으로 HMAC-SHA256 서명을 생성하여 X-Hmac-Signature, X-Hmac-Timestamp, X-Hmac-Nonce 헤더에 담았습니다.

백엔드는 동일한 방식으로 서명을 재계산해 요청이 BFF에서 온 것임을 검증합니다.

Nonce와 타임스탬프를 포함해 서명 재사용(Replay Attack)도 방지했습니다.

Axios 인터셉터에 서명 로직을 주입해 모든 요청에 자동으로 적용되도록 설계했습니다.

WebSocket 중계 – 인증 토큰 노출 방지

실시간 협업 기능의 WebSocket 연결도 동일한 문제가 있었습니다. 프론트엔드가 백엔드 WebSocket에 직접 연결하면 인증 토큰을 URL 쿼리스트링에 노출해야 하는 보안 문제가 발생합니다.

BFF의 WebSocket Gateway가 프론트엔드 연결을 받은 뒤, 쿠키에서 액세스 토큰을 추출하여 백엔드 WebSocket에 Cookie 헤더로 인증 정보를 담아 연결을 수립합니다.

이후 양방향 메시지를 투명하게 중계하는 브리지 구조로 설계했습니다.

Project

내부 오류 정보 은닉 – BFF 계층의 보안 이점

백엔드가 반환하는 에러 응답에는 내부 구현 상세, 스택 트레이스, 서버 구조를 유추할 수 있는 정보가 포함될 수 있습니다.

이를 프론트엔드에 그대로 전달하면 공격자가 내부 구조를 파악하는 데 활용할 수 있습니다.

BFF가 중간에서 모든 에러를 가로채 프로덕션 환경에서는 백엔드의 상세 오류 정보를 제거하고 클라이언트에게는 표준화된 오류 코드와 메시지만 전달합니다.

이처럼 BFF는 HMAC 서명으로 요청 출처를 검증하는 것뿐만 아니라, 응답 방향에서도 내부 정보가 외부로 유출되지 않도록 차단하는 보안 경계선 역할을 합니다.

Work Experience

모바일앱개발협동조합 인턴

커먼플러스 프론트엔드 개발 | 2024.10 - 2025.01

기술 스택: Next.js, TypeScript, Zustand, Storybook, CSS Module

팀 규모: 4명 (프론트 3, 백엔드 1)

리팩토링 주도

인턴을 시작하고 프로젝트에 투입된 후 코드 분석부터 시작하였습니다.

상태관리가 많이 얹혀있고, 중복되는 코드가 많은데, 폴더 구조도 명확하지 않아 파악하기가 많이 어려웠습니다.

인원이 교체된 후에 나와 같은 문제를 겪을 사람이 생길 수도 있고, 당장 작업을 진행하기에도 잠재적인 문제를 꺼안고 시작하는 것보다 리팩토링을 진행 후에 시작하는 것이 생산성이 좋다고 판단하였습니다.

디자인 시스템 구축

우선적으로 피그마 디자인을 확인 하였습니다.

디자인이 정리가 잘되어 있어서 버튼, 텍스트필드 등 작은 단위부터 폼, 탭 레이아웃, 페이지 등 레고처럼 하나씩 조립을 하면서 제작을 할 수 있었고, 이에 가장 적합한 아토믹 디자인 패턴을 적용하기 시작했습니다.

5가지 depth로 atoms → molecules → organisms → templates → pages 순으로 조합을 시작하여 중복된 코드가 약 70% 줄고 빠르게 디자인을 확립할 수 있었습니다.

Atom 테스트

아토믹 디자인 패턴이 성공적으로 적용하려면, 가장 작은 단위인 'Atom'이 모든 상황에서 신뢰할 수 있게 작동해야 합니다.

예를 들어, Button같은 경우

variant: primary, secondary, danger

size: small, medium, large

state: default, hover, disabled, loading

형태의 속성들을 가지고 있을 때, 애플리케이션 전체에서 이 버튼을 테스트하는 것은 거의 불가능했습니다.

때문에 컴포넌트 테스트를 위한 방식을 고민하다 Storybook을 도입했습니다.

Storybook은 완벽한 격리 환경을 제공하고, 모든 상태와 변형에 대한 시각적 테스트를

제공하여 신뢰할 수 있는 Atom을 확보할 수 있게 되었습니다.

또한, 디자이너 팀원분들에게 '살아있는 문서' 역할을 하여 소통이 원활하게 되는 효과도 생겼습니다.

중복 API 요청

API 추가, 수정이 진행되면서 적용하던 중에 네트워크 사용량에 이상을 감지했습니다.

확인해 보니 같은 API를 여러 번 요청하는 부분들이 많이 발견되었습니다.

원인을 따라가 보니 React Hook의 무분별한 사용으로 인하여 같은 함수가 여러 번 호출되는 것을 확인하였습니다.

이에 따라 중복으로 요청되던 API 요청을 해결하기 위해, 상태 관리 또한 다른 방식을 고안하게 되었습니다.

상태 관리 라이브러리

사용자가 등록한 상품의 정보를 여러 페이지, 모달 등에서 사용하는 특성상 전역에서 상태 관리를 진행하는 게 적합했고, 보일러 플레이트가 적은 Zustand를 도입하였습니다.

쪼개져 있던 상태 관리를 Zustand를 통해서 전역에서 전체적으로 관리하고, async action을 통해 store 내부에서 필요한 경우만 fetch를 진행하였습니다.

상태 관리를 명확하게 하였더니 중복 요청되던 API 요청이 없어져, API 호출량이 약 30% 감소하였습니다.

또한, 무분별하게 사용되던 React Hook이 없어지면서 리렌더링 횟수 또한 약 20% 감소하게 되었습니다.